

Deploying a web-app on AWS

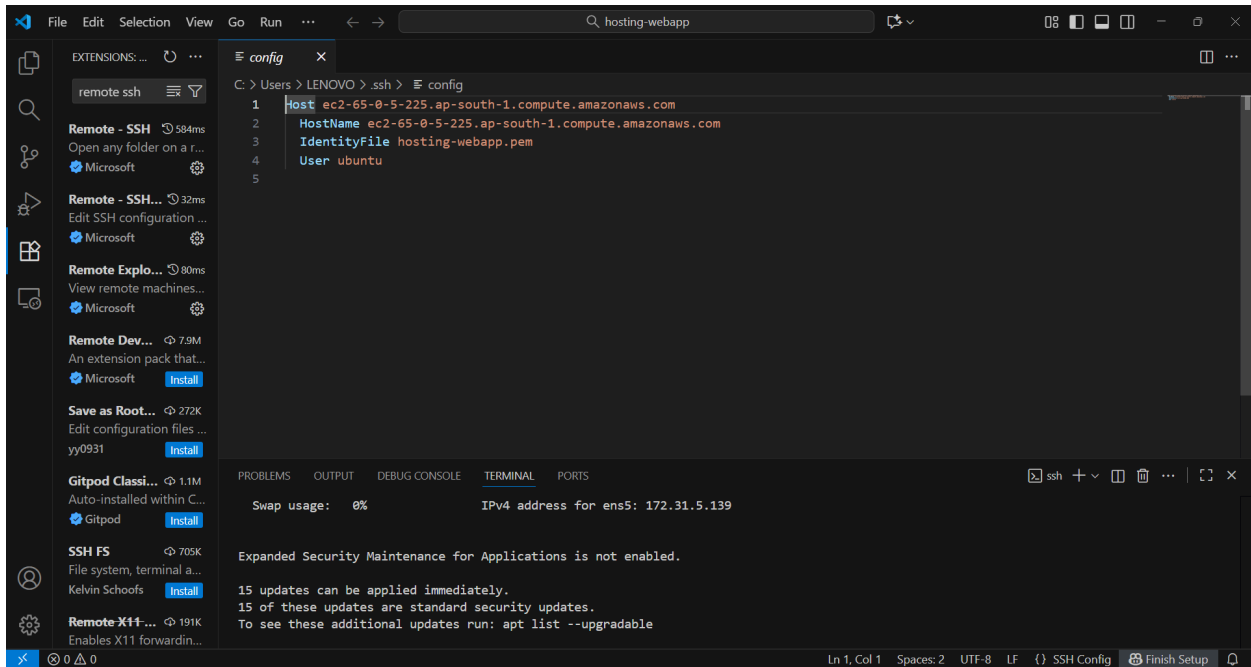
Reff: [NextWork](#)

Creds: Google sign-in: official.atindro@gmail.com

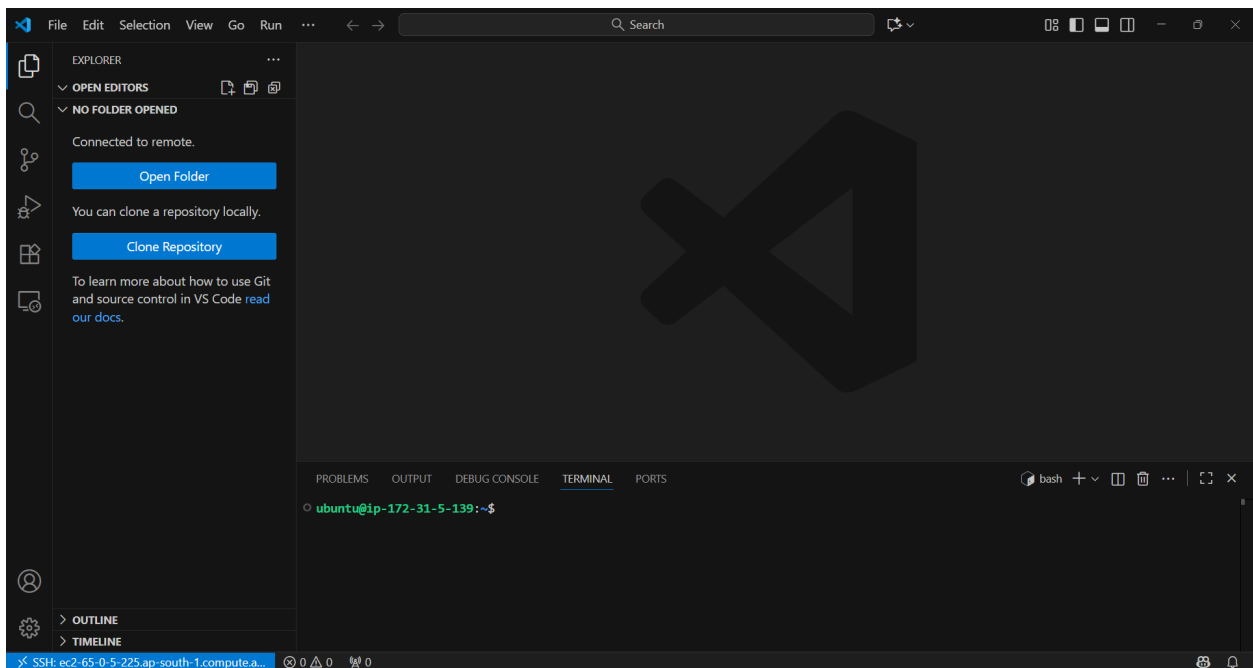
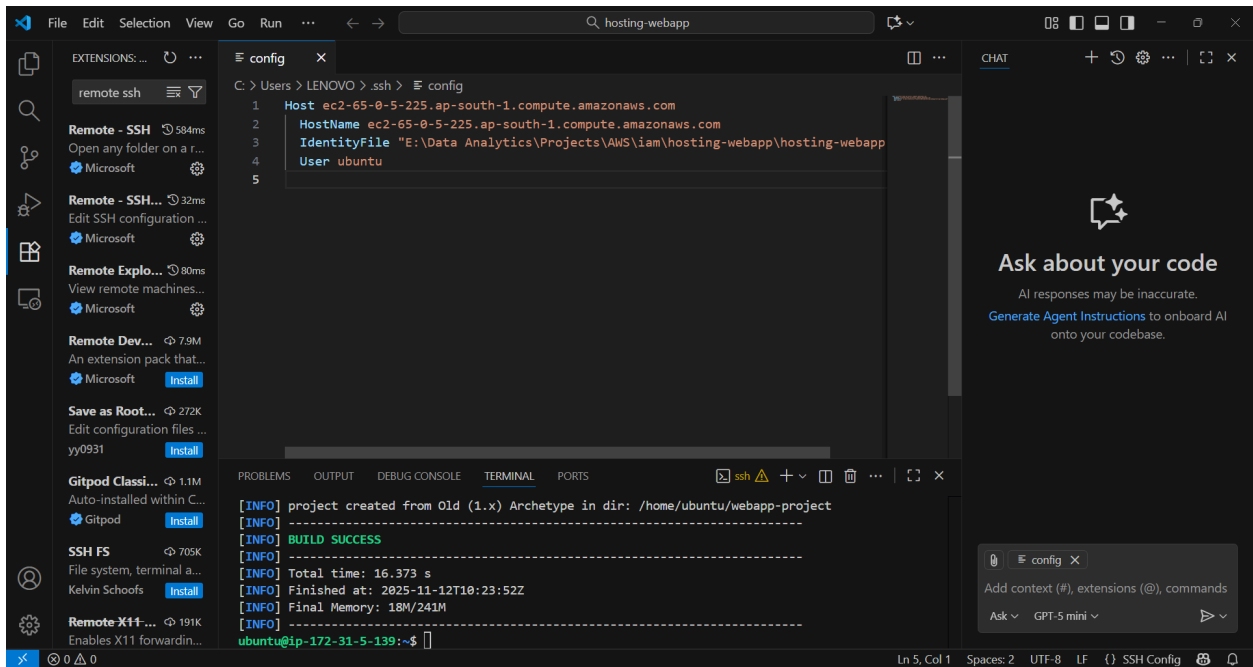
STEPS:

- Make an IAM User: “omgA3dro-IAM-devops”
- Launch ec2, set up the key pair, set up the network configurations
 - Using “ap-south-1” or Mumbai as the server host
 - Creating a Key-Value Pair
 - "E:\Data Analytics\Projects\AWS\IAM Creds\devops\1-project-webapp\deploy-webapp.pem"
 - Setting up the network config
 - Allow SSH traffic from our IP
 - If our IP address is different from what's under My IP, select Custom from the dropdown instead. Enter our IP and make sure to add a /32 to the end e.g. 012.345.678.9/32
- Connect via ssh
 - `ssh -i <.pem file> ubuntu@<publicipv4DNS>`
- Install Dependencies
 - After connecting to our DNS, Install & Update dependencies:
 - apt: `sudo apt update -y`
 - Java SDK: `sudo apt install -y openjdk-11-jdk`
 - Py and pip: `sudo apt install -y python3-pip python3-venv`
 - Download Maven 3.5.2
 - `wget`
<https://archive.apache.org/dist/maven/maven-3/3.5.2/binaries/apache-maven-3.5.2-bin.tar.gz>
 - Extract it to “opt”
 - `sudo tar -xzf apache-maven-3.5.2-bin.tar.gz -C /opt`
 - Add a path to maven
 - `echo "export PATH=/opt/apache-maven-3.5.2/bin:$PATH" >> ~/.bashrc`
 - `source ~/.bashrc`
 - (NOTE: there should be no space between ~ and /, unlike above which have to be made to ensure formatting else an emoji is being formed)

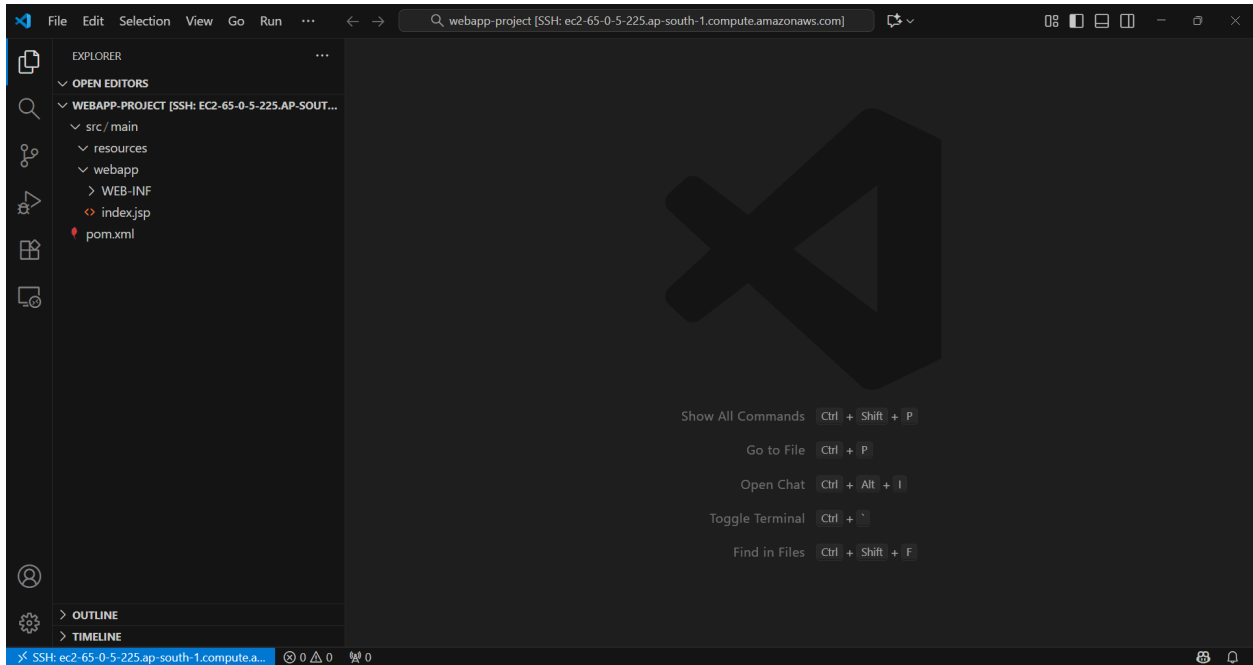
- Select Connect to Host
- Select + Add New SSH Host
- Enter the same ssh
 - `ssh -i <.pem file> ubuntu@<publicipv4DNS>`
 - Select the config file from the drop down and open it



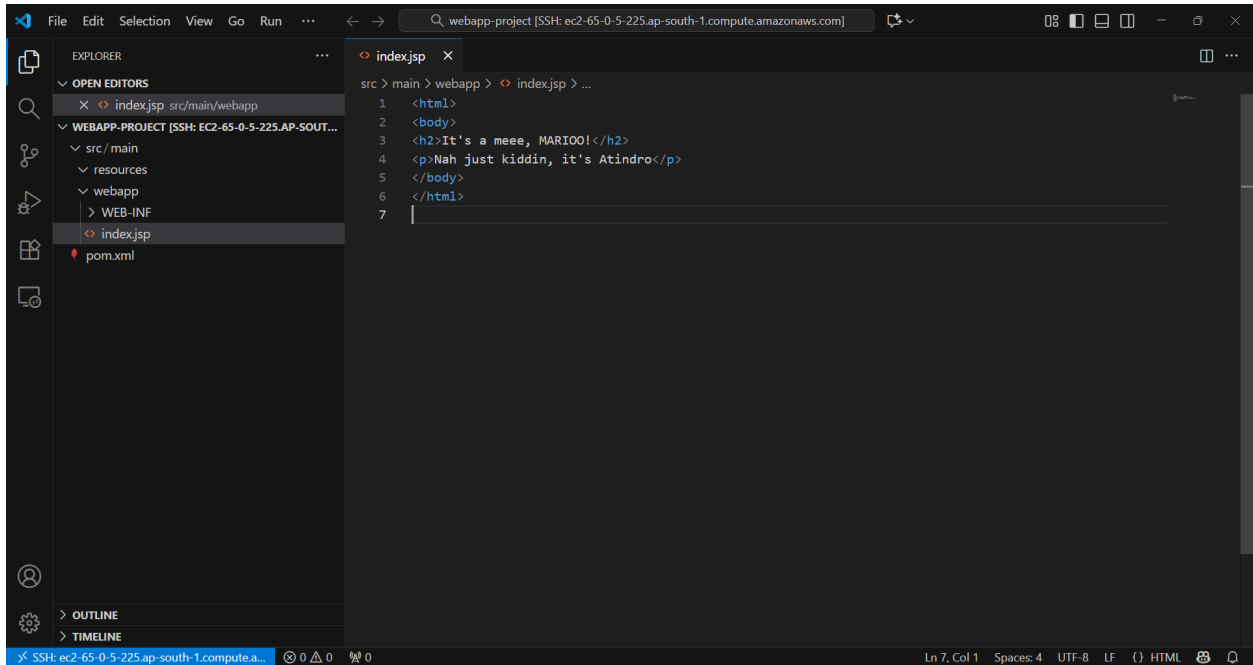
- Now we're ready to connect VS Code with our EC2 instance!
- Click on the double arrow button on the bottom left corner and select Connect to Host again.
- we should now see our EC2 instance listed at the top.
- `Ctrl + Shift + P` and then type `remote ssh - connect`
- Select which platform we prefer
- If we run into any error change the configure file accordingly, ex: fix the file path to our `.pem` file within `"`, user should be `ubuntu` if `ubuntu` is used etc.



- Open Project
 - Select Explorer option on the left window pane
 - Open Project
 - Select the one with our project name which we had given
 - `/home/ubuntu/webapp-project/`
 - Click OK

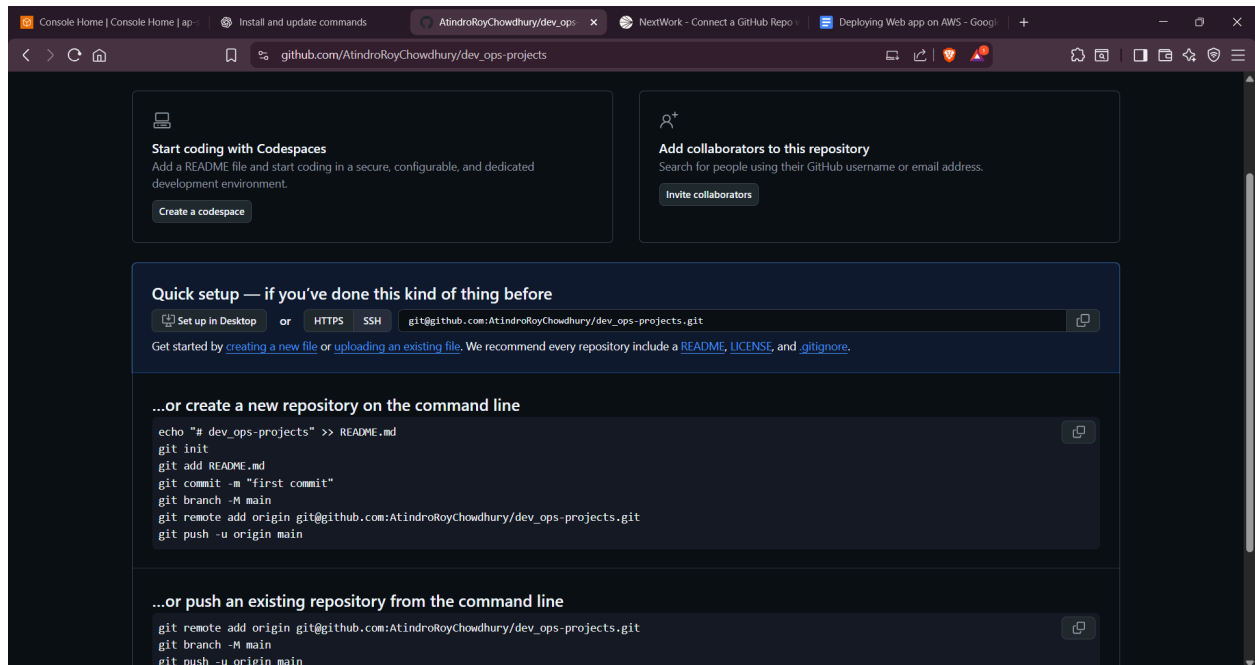


- Info on the directory
 - The src (source) folder holds all the source code files that define how our web app looks and works.
 - src is further divided into webapp, which are the web app's files e.g. HTML, CSS, JavaScript, and JSP files, and resources, which are the configuration files a web app might need e.g. connection settings to a database.
 - pom.xml is a Maven Project Object Model file. It stores information and configuration details that Maven will use to build the project. We'll use pom.xml later in this project series
- Edit on the index.jsp as required



```
src > main > webapp > index.jsp > ...
1  <html>
2  <body>
3  <h2>It's a meee, MARIOO!</h2>
4  <p>Nah just kiddin, it's Atindro</p>
5  </body>
6  </html>
7  |
```

- And then save it
- Now we install git
 - `sudo apt update -y`
 - `sudo apt upgrade -y`
 - `sudo apt install git -y`
 - `git -version`
- Even though our code is on a cloud server like EC2, GitHub helps us use Git and see our file changes in a more user-friendly way. It's just like how using an IDE (VSCode) makes editing code easy.
- Create a repo on Github which we need to connect to our instance.



We created a GitHub repo named: dev_ops-projects

This repo will contain all our DevOps-related folders, including webapp-project.

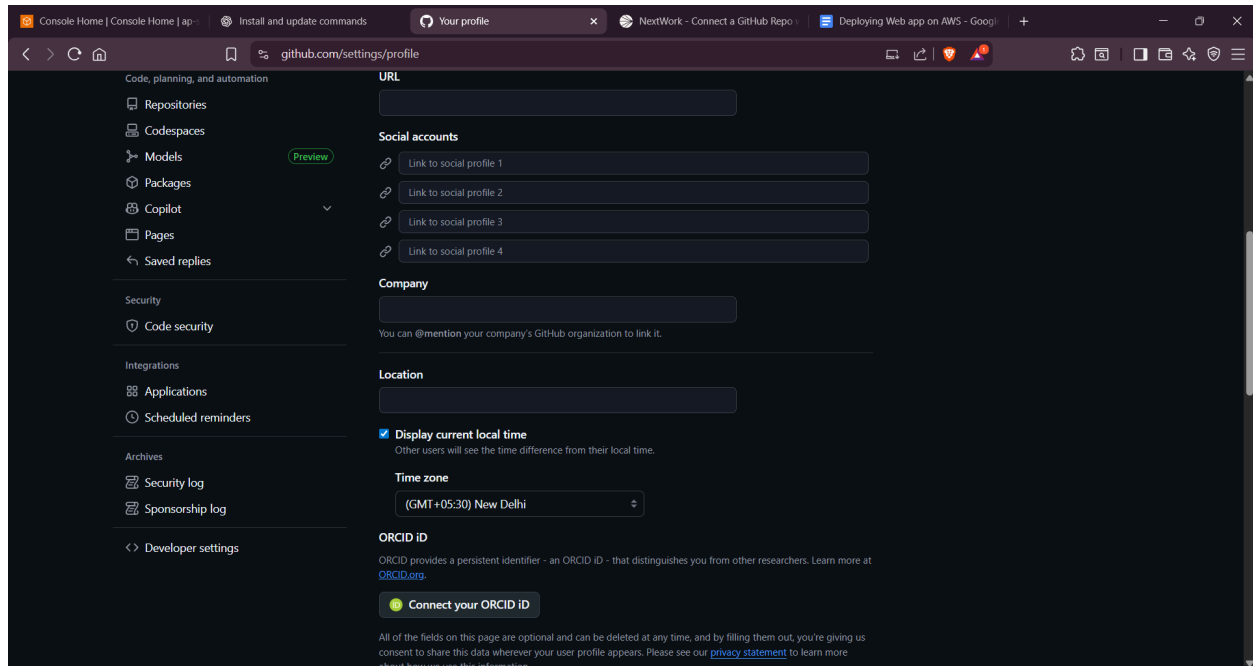
- Lets connect our GitHub repo with our web app project stored in our EC2 instance.
 - Set up a local git repo in our web app folder.
 - Connect our local repo with our GitHub repo.
- Go to our project folder in the instance via the terminal
- To start using Git for our project, we need to create a local repository on our computer. When we run git init inside a directory e.g. webapp-project, it sets up the directory as a local Git repository which means changes are now tracked for version control.
 - git init
- What's a local repository?
 - The local repository is where we use Git directly on our own EC2 instance. The edits we make in our local repo are only visible to us and aren't shared with anyone else yet. This is different to the GitHub repository, which is the remote/cloud version of our repo that others can see.
- What is a main branch?
 - we can think of Git branches as parallel versions or 'alternate universes' of the same project. For example, if we wanted to test a change to our code, we can set up a new branch that lets us diverge from the original/main version of our code (called master) so we can experiment with new features or test bug fixes safely. We won't create new branches in this project and we'll save all new changes

directly to master, but it's best practice to make all changes in a separate branch and then merge them into master when they're ready.

- Now copy the https url from the github quick setup
- Our local and GitHub repositories aren't automatically linked, so we'll need to connect the two so that updates made in our local repo can also reflect in our GitHub repo.
 - Hence we'll use "remote add origin"
- Paste this url below with this code
 - `git remote add origin <github https quick setup link>`
Example of the setup link:
"[git@github.com:AtindroRoyChowdhury/dev_ops-projects.git](https://github.com/AtindroRoyChowdhury/dev_ops-projects.git)"
- Clone it:
 - `git clone https://github.com/AtindroRoyChowdhury/dev_ops-projects.git`
- Move our project into the repository
 - Our web application code existed locally on the server inside:
 - `~ /webapp-project`
 - Move it inside the cloned Git repo:
 - `mv ~ /webapp-project ~ /dev_ops-projects/`
- Stage and commit the project
 - Go inside the repo:
 - `cd ~ /dev_ops-projects`
 - Add all files:
 - `git add .`
 - Commit
 - `git commit -m "Added webapp-project folder"`
- Connect to GitHub using HTTPS + Personal Access Token
 - `git remote -v`
 - We'll get some .git fetch and pull result, if not then do this
 - `git remote set-url origin`
https://github.com/AtindroRoyChowdhury/dev_ops-projects.git
 - When we push, Git will ask for:
 - Username: our GitHub username
 - Password: our GitHub Personal Access Token (PAT)
 - Push to github
 - `git push origin main`

If we wish to connect via SSH:

- Now generate github cred and save it with we only if we want to connect using SSH

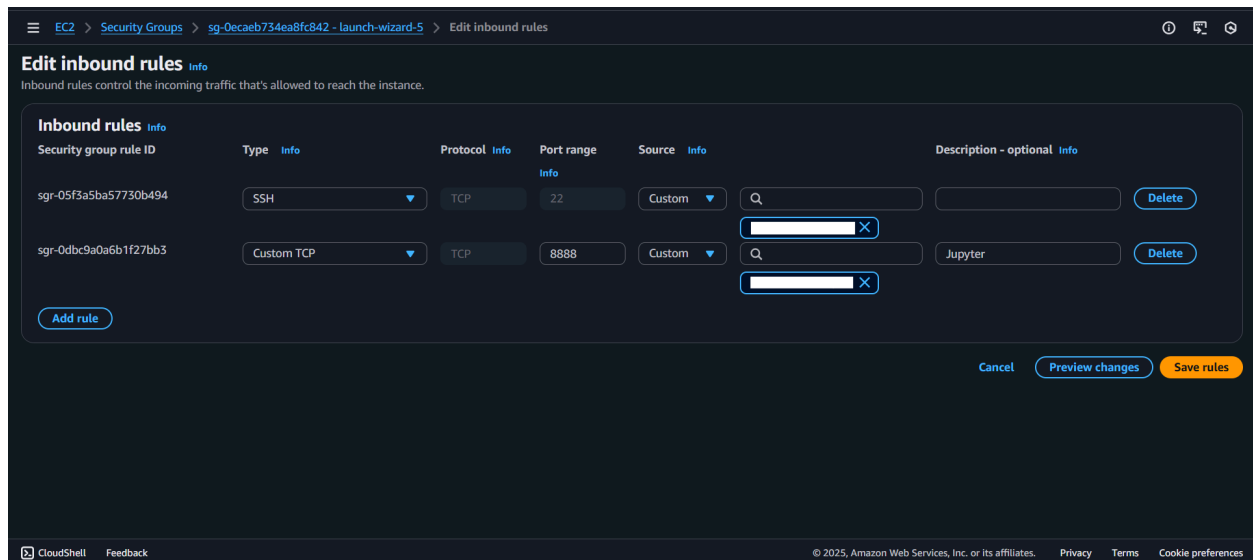


- Select Developer Settings
 - Select Personal access tokens.
 - Select Tokens (classic).
 - Select Generate new token.
 - Select Generate new token (classic).
- Note: Mine was generated for a time period of 90 days, I recommend we lowering it down to 7 for more security.
- - `git push -u origin master`
 - **What do these commands do?**
 - `git commit -m "Updated index.jsp with new content"` saves the staged changes as a snapshot in our project's history. This means our project's version control history has just saved our latest changes in a new version. `-m` flag lets us leave a message describing what the commit is about, making it easier to review what changed in this version.
 - `git push -u origin master` uploads i.e. 'pushes' our committed changes to origin, which we've bookmarked as our GitHub repo. 'master' tells Git that these updates should be pushed to the master branch of our GitHub repo. By using `-u` we're also setting an 'upstream' for our local branch, which means we're telling Git to remember to push to master by default.

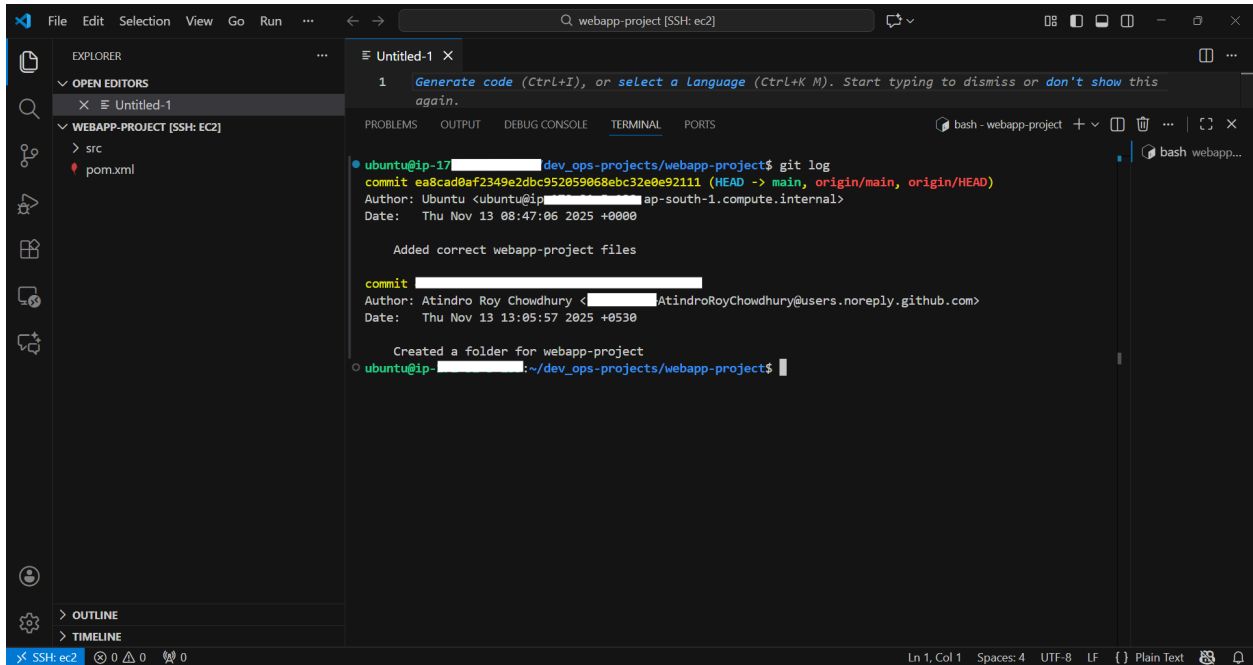
Next time, we can simply run git push without needing to define origin and master.

- CONCEPTS:
 - Local Repository:
 - This is the Git repo stored on our EC2 instance (dev_ops-projects).
 - All changes we make here are private until we push them to GitHub.
 - Remote Repository:
 - This is the GitHub version of the repo.
 - It is the shared/cloud copy.
 - Main Branch:
 - The main branch is the default version of the project.
 - We could create feature branches later, but for this project we directly commit to main.

NOTE: If the ssh refuses to connect, check if our ip is matching with the instance's security group's ip configurations.



- Let's check the git logs to see if the commit was successful via our remote server
 - git log



The screenshot shows a VS Code interface with a terminal window open. The terminal displays the output of a `git log` command. The output shows two commits. The first commit is a merge commit: `commit ea8cad0af2349e2dbc952059068ebc32e0e92111 (HEAD -> main, origin/main, origin/HEAD)`. The second commit is a regular commit: `commit [redacted]`. The terminal also shows the message "Added correct webapp-project files" and "Created a folder for webapp-project". The terminal prompt is `ubuntu@ip-172-31-1-10: ~/dev_ops-projects/webapp-project$`.

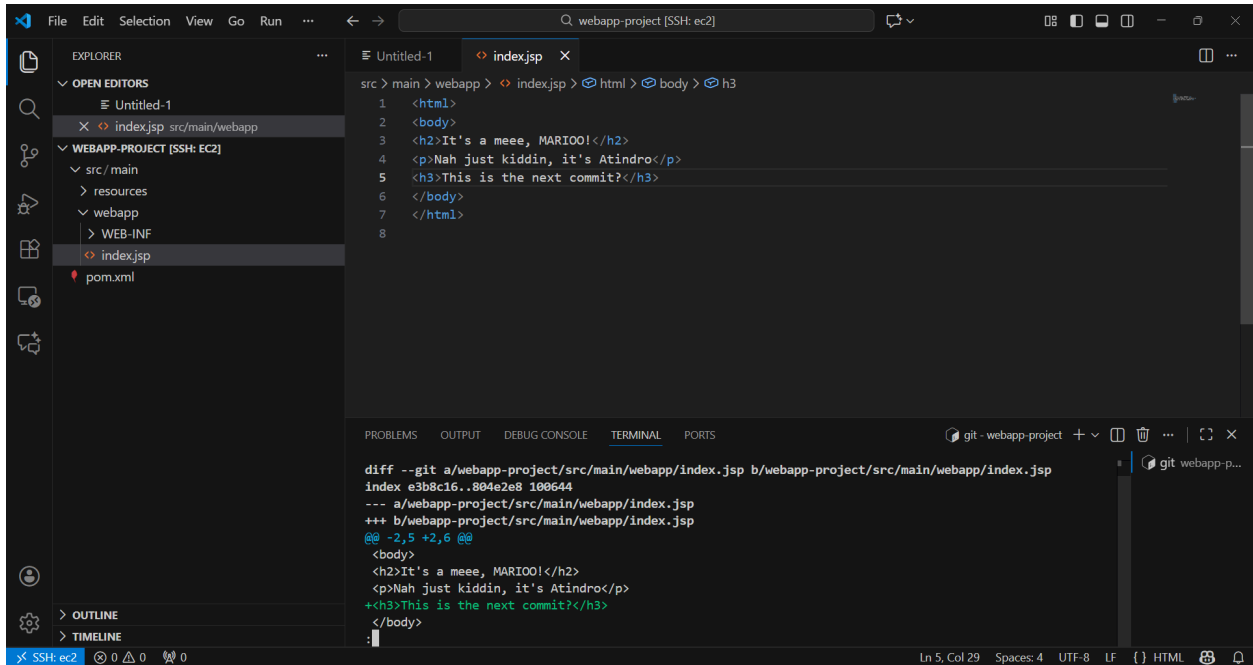
```
ubuntu@ip-172-31-1-10:~/dev_ops-projects/webapp-project$ git log
commit ea8cad0af2349e2dbc952059068ebc32e0e92111 (HEAD -> main, origin/main, origin/HEAD)
Author: Ubuntu <ubuntu@ip-172-31-1-10.ap-south-1.compute.internal>
Date: Thu Nov 13 08:47:06 2025 +0000

    Added correct webapp-project files

commit [redacted]
Author: Atindro Roy Chowdhury <[redacted]@users.noreply.github.com>
Date: Thu Nov 13 13:05:57 2025 +0530

    Created a folder for webapp-project
ubuntu@ip-172-31-1-10:~/dev_ops-projects/webapp-project$
```

- We were successful as we can see here:
 - We are on the correct directory
 - HEAD/main: Our current local branch is main
 - origin/main: Our remote (GitHub) also has main
 - origin/HEAD: GitHub's default branch is main
 - Everything is synced and clean.
 - The two commits were also successful
- Now we will make changes to the web-app and commit the changes.
- Make changes in the index.jsp file and then
 - `git add .`
 - `git diff --staged`

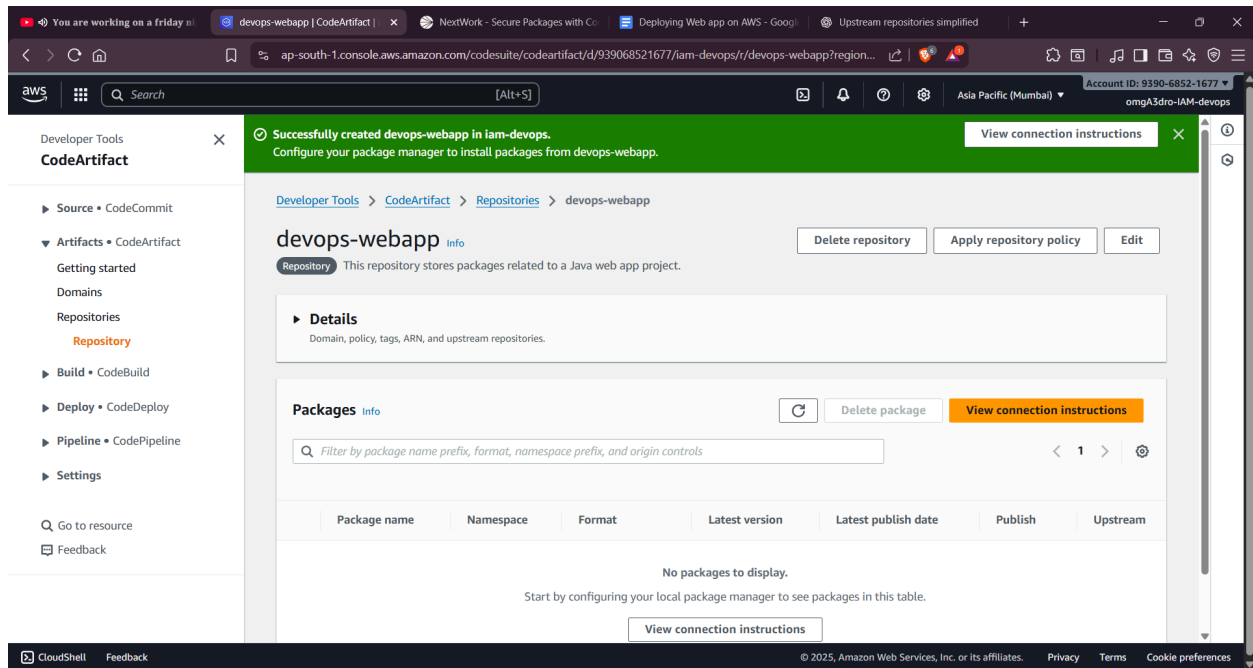


- This will show we the changes made in green as we see above.
- Press q to quit the view then
 - git commit -m "Add new line to index.jsp"
 - git push
- Now we can see the changes in the github repo too!

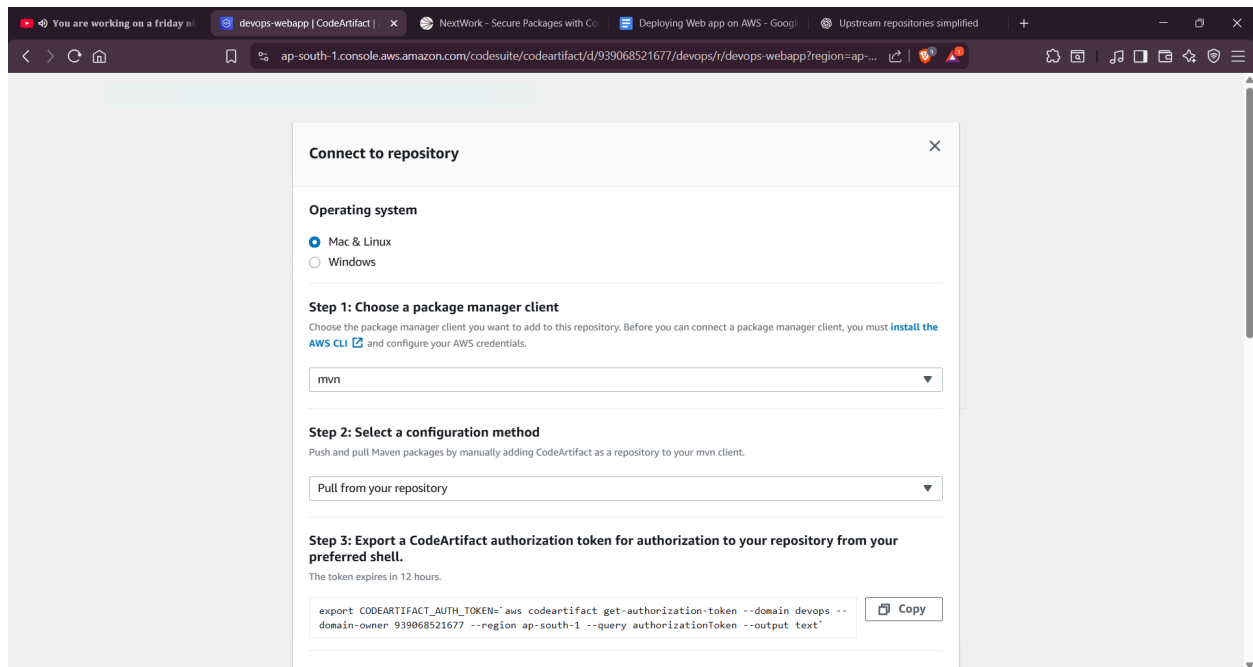
Securing the web application packages via AWS CodeArtifact

- Well the key difference between a github repo and an aws codeartifact is that github acts like a collaboration platform for the source code whereas the codeartifact acts as a private package and library collaboration platform.
- This is important because CodeArtifact provides a centralized, secure, and scalable way to manage dependencies for our Java projects, improving build consistency and security.
- Go to AWS CodeArtifact
 - Select Maven as the upstream repository
 - Give it a repo name
 - Then click next
 - Use on "This AWS Account"
 - Give it a domain name (our team name in general)
 - Click next and then create the repository

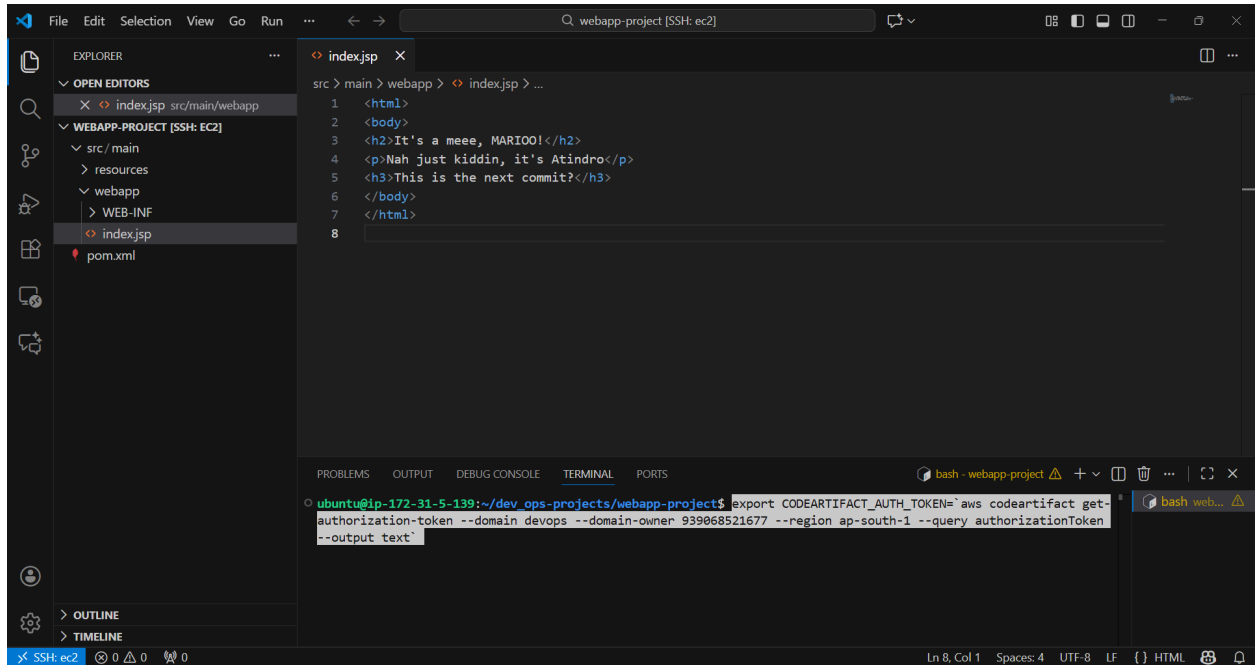
- Click on “View Connections” on the CODEArtifact’s Repositories’s page



- Then make sure these are the connections



- Use Mac & Linux if the EC2 instance is Ubuntu(or Linux)
- We might face an error if we don't have the correct permissions and policies attached to our CodeArtifact domain/repo.
- Copy the command in the step 3 mentioned “Export a CodeArtifact authorization..”
- Paste it in the terminal (VSCode)



- But if we face any error, try these:
 - `curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"`
 - `unzip awscliv2.zip`
 - `sudo ./aws/install`
 - `aws configure`
 - Put our creds (if we forget, create an access key in our IAM dashboard for our "User")
 - IAM
 - Users (On the left pane)
 - Select our user
 - Security
 - Access keys
 - CLI
 - Set the aws configure
 - Now run the said step 3 code as above again
 - IT will run, incase it didn't its some policy issue, create a policy with this json:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
```

```

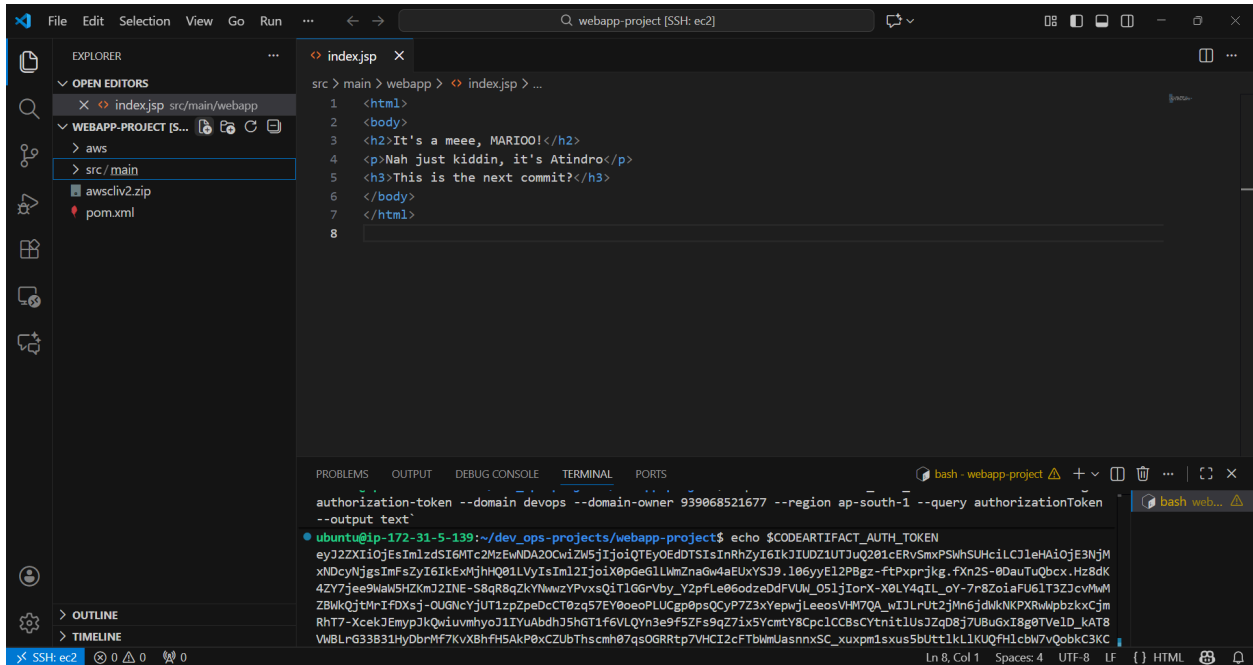
    "Action": [
        "codeartifact:GetAuthorizationToken",
        "codeartifact:GetRepositoryEndpoint",
        "codeartifact:ReadFromRepository"
    ],
    "Resource": "*"
},
{
    "Effect": "Allow",
    "Action": "sts:GetServiceBearerToken",
    "Resource": "*",
    "Condition": {
        "StringEquals": {
            "sts:AWSServiceName": "codeartifact.amazonaws.com"
        }
    }
}
]
}

```

- Now create a role and attach that role to our running EC2 instance using the modify the IAM role in the security of the action aspect of it.
- Anyways, if done successfully we will see a 64 character string when we pass this command in the terminal itself
 - `echo $CODEARTIFACT_AUTH_TOKEN`

Setting up the xml file

- In VS Code, in our left hand file explorer, head to the root directory of the web-project.



The screenshot shows the VS Code interface with a project named 'webapp-project' opened via SSH on an EC2 instance. The Explorer sidebar on the left shows the file structure: 'src/main/webapp' containing 'index.jsp'. The main editor displays the content of 'index.jsp', which is an HTML file with the following code:

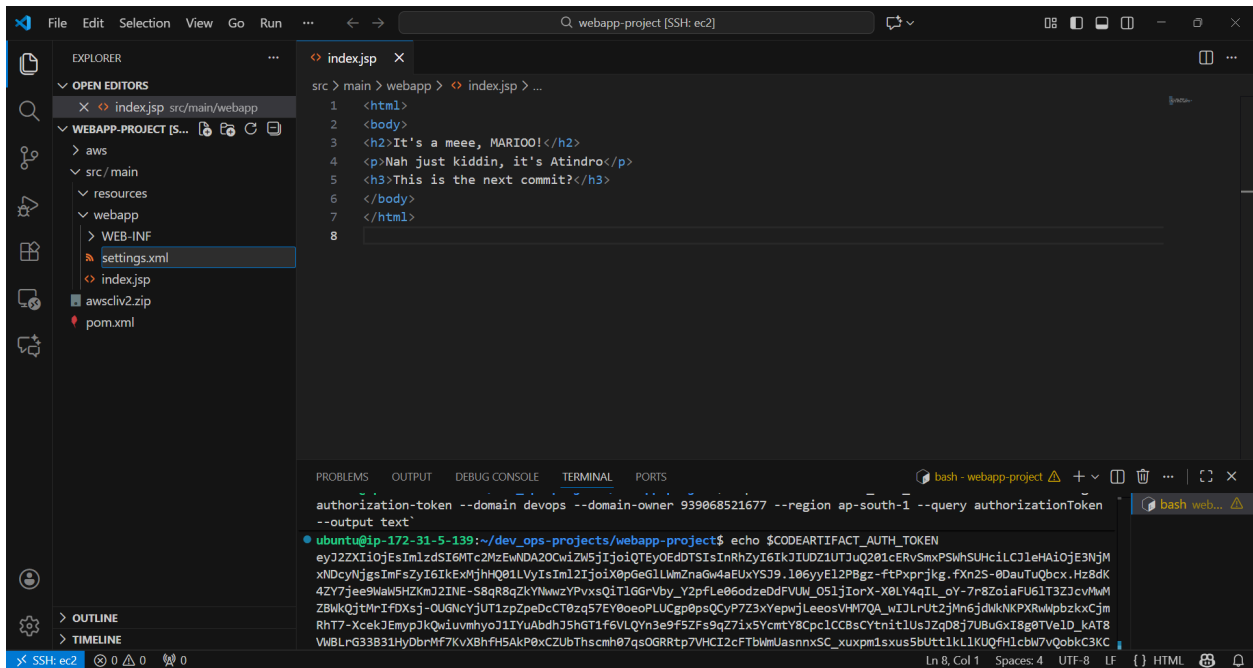
```
1 <html>
2 <body>
3 <h2>It's a meee, MARIOO!</h2>
4 <p>Nah just kiddin, it's Atindro</p>
5 <h3>This is the next commit?</h3>
6 </body>
7 </html>
8
```

The bottom panel shows the TERMINAL with a bash prompt. The current command being executed is:

```
ubuntu@ip-172-31-5-139:~/dev_ops-projects/webapp-project$ echo $CODEARTIFACT_AUTH_TOKEN
```

The terminal output shows a long authorization token.

- Create a new file at the root of the web-project directory.



The screenshot shows the VS Code interface with the same 'webapp-project' project. The Explorer sidebar now shows a new directory 'resources' created under 'src/main/webapp'. Inside 'resources', a new file 'settings.xml' has been created and is selected. The main editor shows the content of 'index.jsp' from the previous screenshot.

The bottom panel shows the TERMINAL with the same bash prompt as before:

```
ubuntu@ip-172-31-5-139:~/dev_ops-projects/webapp-project$ echo $CODEARTIFACT_AUTH_TOKEN
```

- Put this code below and just change the repo url as req

```
<settings>
<servers>
<server>
<id>codeartifact</id>
<username>aws</username>
```

```

    <password>${env.CODEARTIFACT_AUTH_TOKEN}</password>
  </server>
</servers>

<profiles>
  <profile>
    <id>codeartifact-maven</id>
    <repositories>
      <repository>
        <id>codeartifact</id>

        <url>https://devops-939068521677.d.codeartifact.ap-south-1.amazonaws.com/maven/devops-w
ebapp/</url>
      </repository>
    </repositories>
  </profile>
</profiles>

<mirrors>
  <mirror>
    <id>codeartifact</id>
    <name>CodeArtifact Mirror</name>

    <url>https://devops-939068521677.d.codeartifact.ap-south-1.amazonaws.com/maven/devops-w
ebapp/</url>
    <mirrorOf>*</mirrorOf>
  </mirror>
</mirrors>

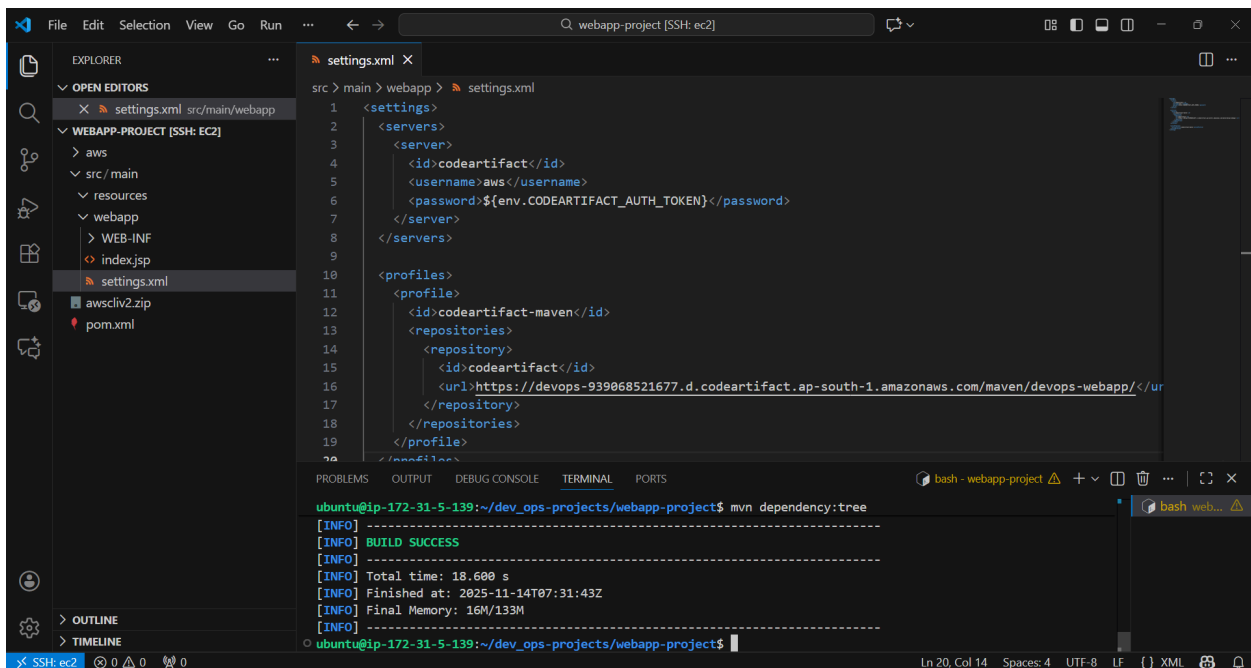
<activeProfiles>
  <activeProfile>codeartifact-maven</activeProfile>
</activeProfiles>
</settings>

```

- We can get our repo url from this command:

```
aws codeartifact get-repository-endpoint \  
--domain devops \  
--domain-owner 939068521677 \  
--repository <our-repo-name> \  
--format maven
```

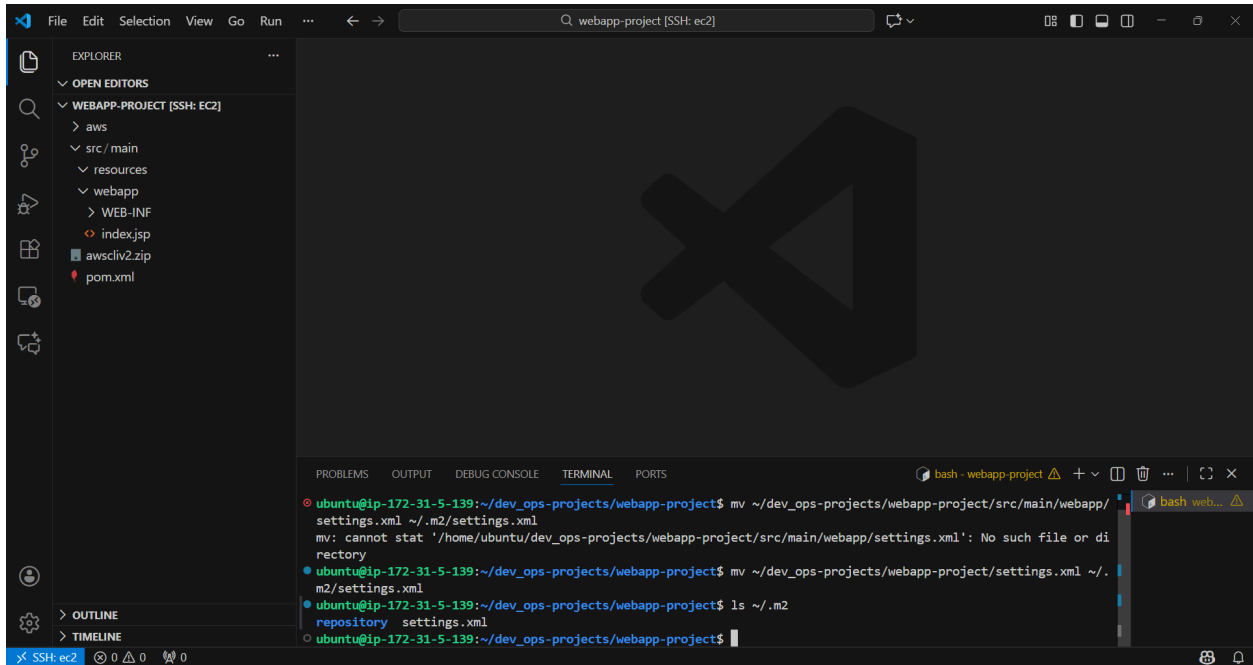
- Now check via this command
 - mvn dependency:tree
- And Voila!



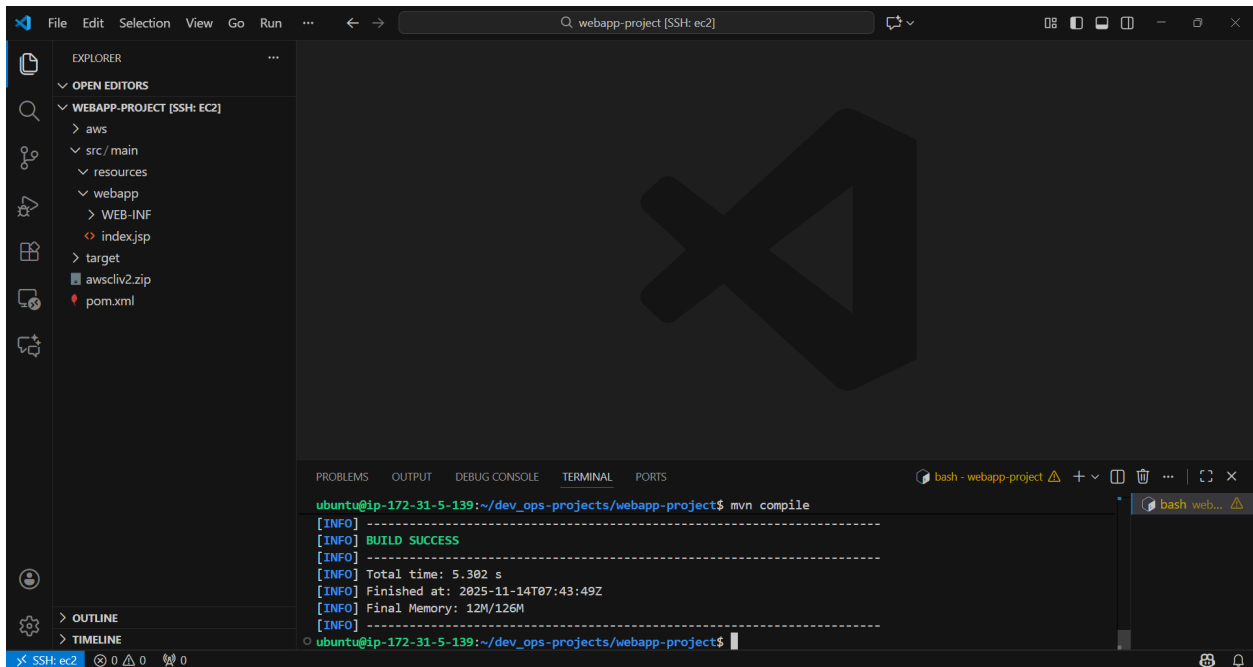
- Now run these commands as maven cant compile files in our repo unless the settings.xml is inside the ~ / m2 folder

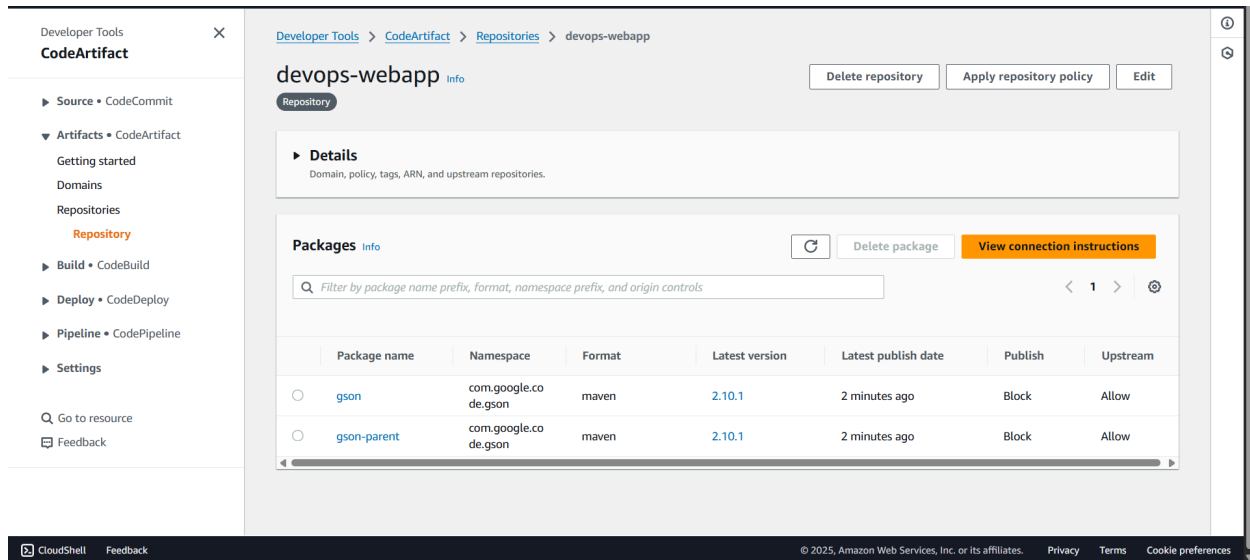
```
mkdir -p ~ /.m2
```

```
mv ~ /dev_ops-projects/webapp-project/settings.xml ~ /.m2/settings.xml
```



- Now we compile it using
 - mvn compile





- Any further dependency or other things in the pom.xml file will reflect here now.
- Whenever we use the code artifact, we must run this as the tokens expire every 12 hours.

```
export CODEARTIFACT_AUTH_TOKEN=$(aws codeartifact get-authorization-token \
--domain devops \
--domain-owner 939068521677 \
--region ap-south-2 \
--query authorizationToken \
--output text)
```

AWS Codebuild

- AWS CodeBuild is a fully built tool for our code. It takes our source code, compiles it, runs tests, and packages it up. We engineers love continuous integration tools like CodeBuild because we don't have to manually set up and manage any build servers ourselves, and we only pay for the compute time we use for building our projects (instead of entire servers that are idle most of the time). Think of it as a super-efficient, scalable, and managed service that handles all the heavy lifting of building and testing our applications.

- Codebuild
 - Build Projects
 - On the **Create build project** page, scroll to the **Project configuration** section.
 - Under **Project name**, enter the name.
 - Under **Project type**, make sure **Default project** is selected.
 - Select Github as the source type

Additional configuration
Description, public build access, build badge, concurrent build limit, tags

▼ Source Add source

Source 1 - Primary

Source provider

- No source
- No source ✓
- GitHub
- GitHub Enterprise Server
- Bitbucket
- GitLab
- GitLab Self Managed
- Amazon S3
- AWS CodeCommit

Use an image managed by AWS CodeBuild Specify a Docker image

Running mode

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- If the github access causes any issue, use the token we used before for authenticating github.

Additional configuration
Description, public build access, build badge, concurrent build limit, tags

▼ Source Add source

Source 1 - Primary

Source provider

Credential

✓ Your account is successfully connected through PAT using CodeBuild managed token. [Manage account credentials.](#)

☐ Use override credentials for this project only

Repository

☒ Repository in my GitHub account ☐ Public repository ☐ GitHub scoped webhook

Q X ↺

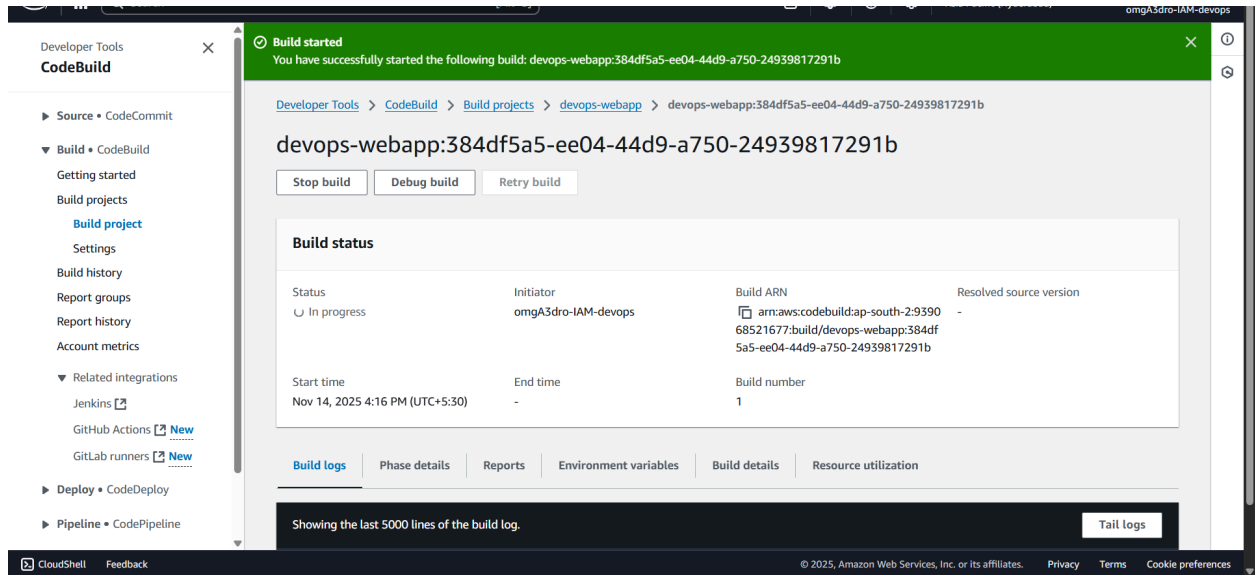
Source version - optional [Info](#)
Enter a pull request, branch, commit ID, tag, or reference and a commit ID.

Additional configuration
Git clone depth, Git submodules, Build status config

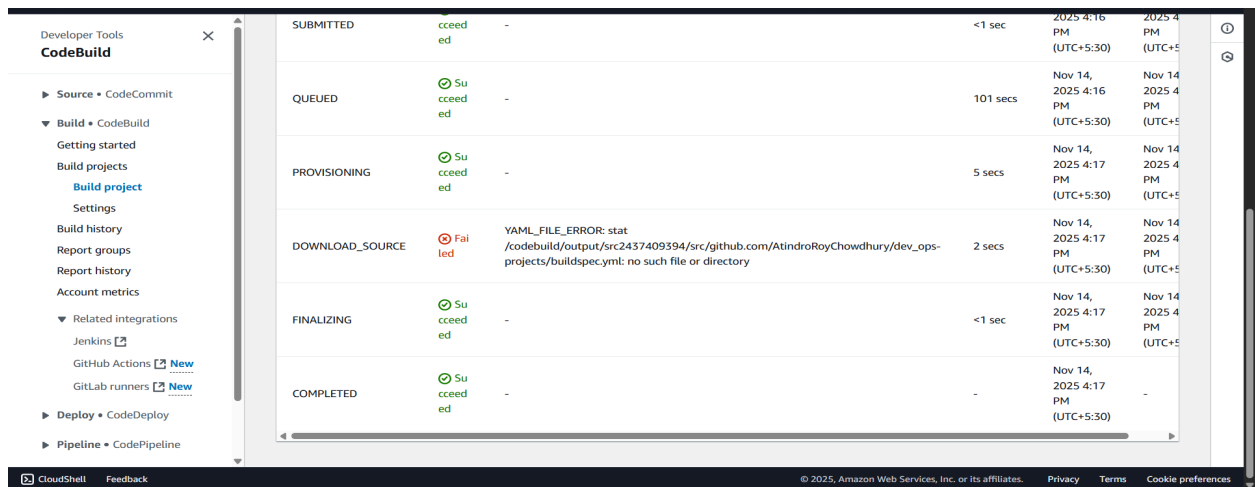
CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Configurations
 - Scroll down to the **Primary source webhook events** section.

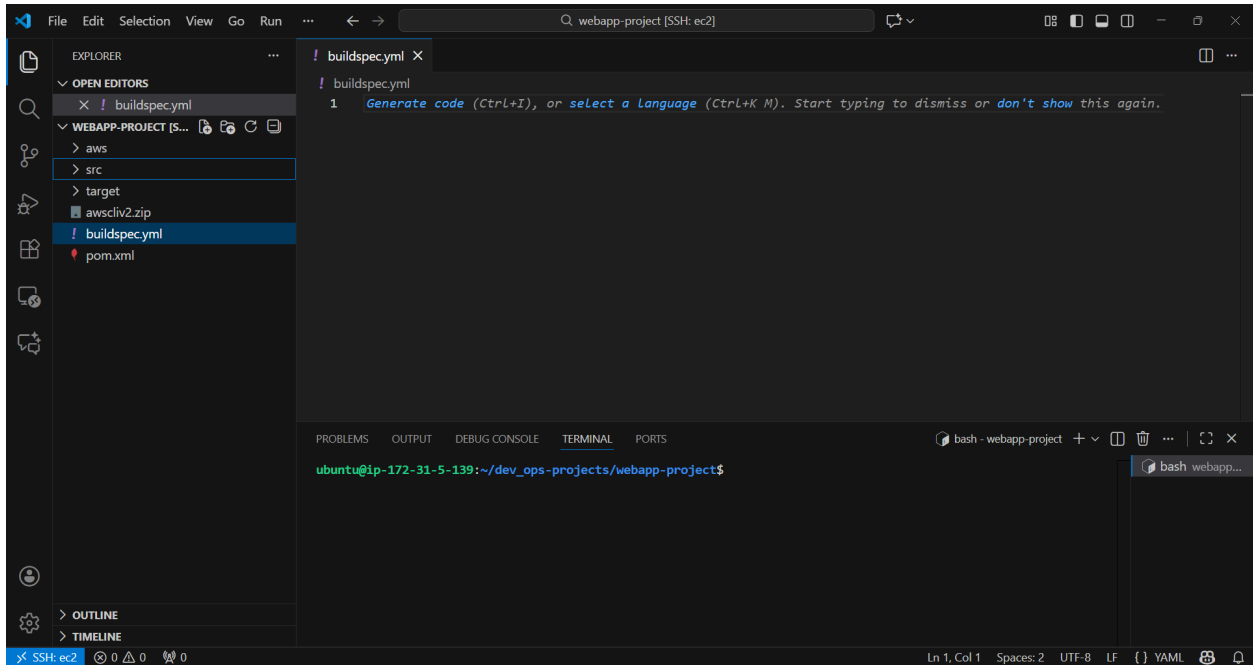
- **Untick** the **Webhook** checkbox that says "Rebuild every time a code change is pushed to this repository." Scroll down to the **Environment** section.
- Under Environment, for Operating system, select Amazon Linux.
- For Runtime(s), select Standard.
- For Image, choose aws/codebuild/amazonlinux-x86_64-standard:corretto8.
- Scroll down to the **Buildspec** section.
- Under **Buildspec format**, select **Use a buildspec file**.
- Give it a name: buildspec.yml
 - NOTE: If the yml file is in the root directory, well and good
 - Else, name it like this
 - webapp-project/buildspec.yml
 - The first part is the folder inside the root directory and then the file. This will give us the edited repo link in the final CI/CD integrations.
- Artifacts 1
 - Amazon S3
 - Make a S3 bucket (on another tab)
 - Select it in the drop down post making
 - Click on the zip option
- For logs
 - Amazon CloudWatch Logs is a monitoring service that collects and tracks logs from AWS services. In this project, CloudWatch will record everything that happens during the build process, including the commands that are run, the output of those commands, and any errors that occur. This is incredibly useful for debugging and understanding what went wrong if a build fails.
 - Put the name: aws/codebuild/devops-webapp
 - Create build project
- Start build



- If we see an error that'll be due to the missing yml file in the github repo



- Create it in the VSCode in the root folder itself



- Paste this

version: 0.2

phases:

install:

runtime-versions:

java: corretto8

pre_build:

commands:

- echo Logging in to AWS CodeArtifact...

- CODEARTIFACT_AUTH_TOKEN=`aws codeartifact get-authorization-token --domain nextwork --domain-owner <your\_aws\_account\_id> --region <your aws region>--query authorizationToken --output text`

- export CODEARTIFACT_AUTH_TOKEN

build:

commands:

- echo Build started on `date`

- mvn clean install -s settings.xml

post_build:

commands:

- echo Build completed on `date`

- echo Packaging artifacts...

- mvn package -s settings.xml

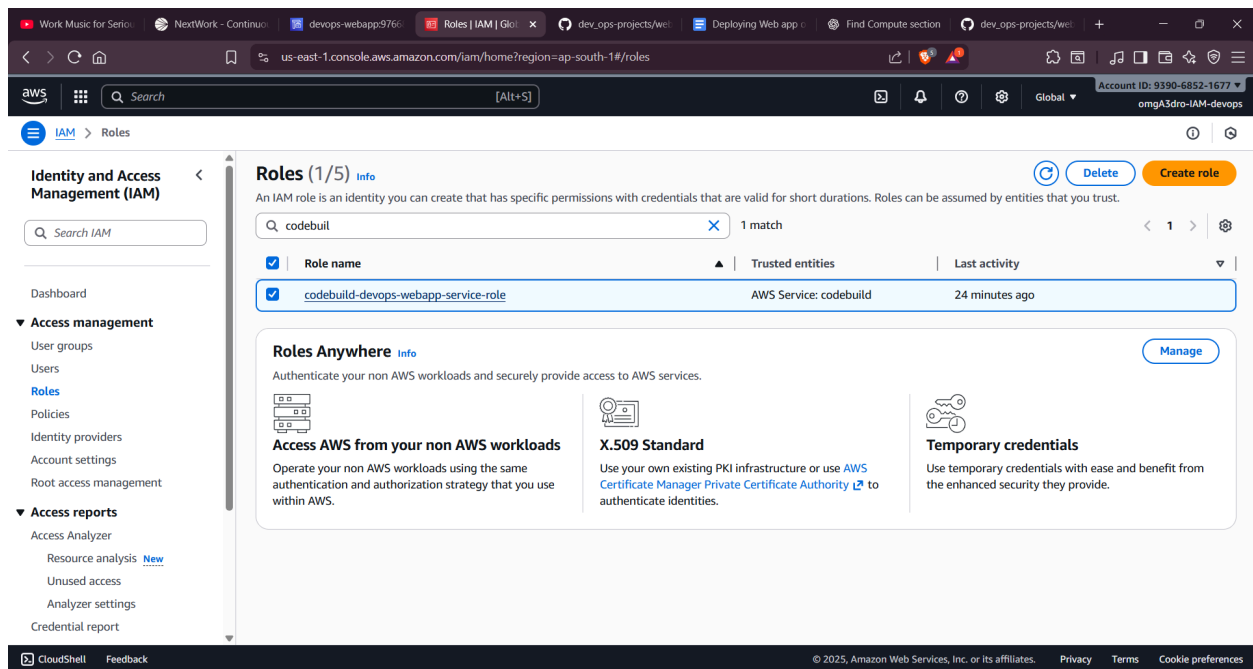
artifacts:

files:

- target/*.war

discard-paths: no

- Add, commit and push
 - git add .
 - git commit -m "Adding buildspec.yml file"
 - git push
- Also safecase: Add IAM roles (imp)



- Click on it and make a new policy
 - Use the json that we kept for safe keeping for policy before? Use that

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "codeartifact:GetAuthorizationToken",
```

```

        "codeartifact:GetRepositoryEndpoint",
        "codeartifact:ReadFromRepository"
    ],
    "Resource": "*"
  },
  {
    "Effect": "Allow",
    "Action": "sts:GetServiceBearerToken",
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "sts:AWSServiceName": "codeartifact.amazonaws.com"
      }
    }
  }
}
]
}

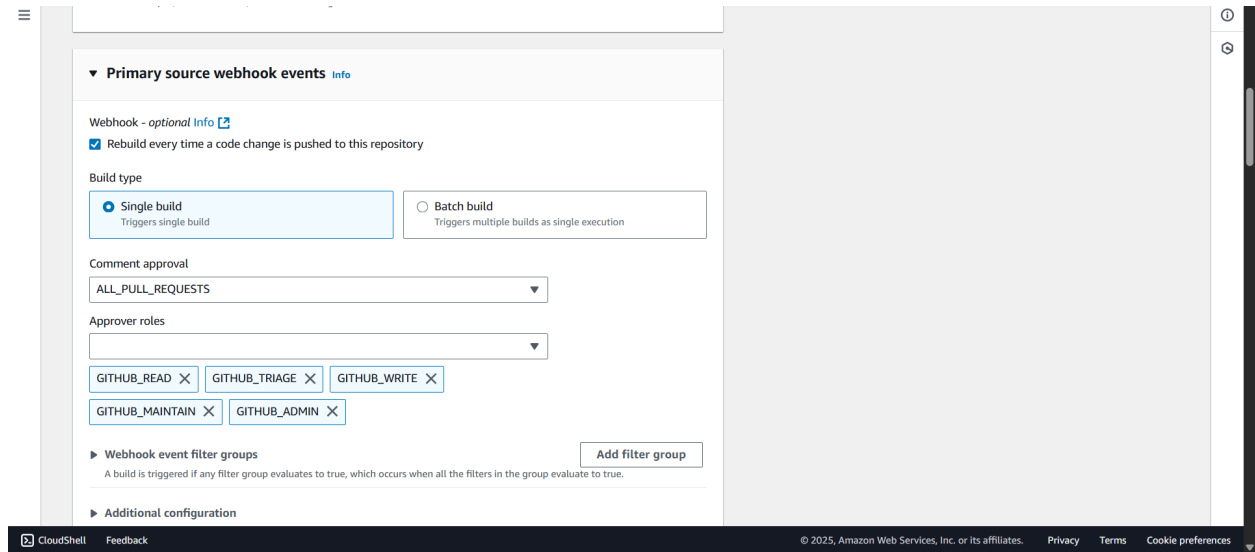
```

- Retry the build on the AWS CodeBuild

The screenshot shows the AWS CodeBuild console interface. At the top, there are two tabs for build logs: "Nov 14, 2025 6:18 PM (UTC+5:30)" and "Nov 14, 2025 6:19 PM (UTC+5:30)". Below the tabs is a navigation bar with links: "Build logs", "Phase details" (which is active), "Reports", "Environment variables", "Build details", and "Resource utilization". The main content area displays a table of build phases. All phases are marked as "Succeeded" with a green checkmark icon. The phases include SUBMITTED, QUEUED, PROVISIONING, DOWNLOAD_SOURCE, INSTALL, PRE_BUILD, BUILD, POST_BUILD, UPLOAD_ARTIFACTS, FINALIZING, and COMPLETED. The BUILD phase is the longest, taking 42 seconds. The bottom of the screen shows a footer with "CloudShell", "Feedback", and copyright information for Amazon Web Services.

Name	Status	Context	Duration	Start time	End time
SUBMITTED	✔ Succeeded	-	<1 sec	Nov 14, 2025 6:18 PM (UTC+5:30)	Nov 14, 2025 6:18 PM (UTC+5:30)
QUEUED	✔ Succeeded	-	<1 sec	Nov 14, 2025 6:18 PM (UTC+5:30)	Nov 14, 2025 6:18 PM (UTC+5:30)
PROVISIONING	✔ Succeeded	-	7 secs	Nov 14, 2025 6:18 PM (UTC+5:30)	Nov 14, 2025 6:18 PM (UTC+5:30)
DOWNLOAD_SOURCE	✔ Succeeded	-	11 secs	Nov 14, 2025 6:18 PM (UTC+5:30)	Nov 14, 2025 6:18 PM (UTC+5:30)
INSTALL	✔ Succeeded	-	<1 sec	Nov 14, 2025 6:18 PM (UTC+5:30)	Nov 14, 2025 6:18 PM (UTC+5:30)
PRE_BUILD	✔ Succeeded	-	8 secs	Nov 14, 2025 6:18 PM (UTC+5:30)	Nov 14, 2025 6:18 PM (UTC+5:30)
BUILD	✔ Succeeded	-	42 secs	Nov 14, 2025 6:18 PM (UTC+5:30)	Nov 14, 2025 6:19 PM (UTC+5:30)
POST_BUILD	✔ Succeeded	-	2 secs	Nov 14, 2025 6:19 PM (UTC+5:30)	Nov 14, 2025 6:19 PM (UTC+5:30)
UPLOAD_ARTIFACTS	✔ Succeeded	-	<1 sec	Nov 14, 2025 6:19 PM (UTC+5:30)	Nov 14, 2025 6:19 PM (UTC+5:30)
FINALIZING	✔ Succeeded	-	<1 sec	Nov 14, 2025 6:19 PM (UTC+5:30)	Nov 14, 2025 6:19 PM (UTC+5:30)
COMPLETED	✔ Succeeded	-	-	Nov 14, 2025 6:19 PM (UTC+5:30)	-

- Lets automate the Continuous Integration
 - Edit the Codebuild



And Voila!

Automating and setting up the deployment

- Run these in the same instance

```
sudo apt update -y
```

```
sudo apt install ruby wget -y
```

```
cd /home/ubuntu
```

```
wget https://aws-codedeploy-ap-south-1.s3.ap-south-1.amazonaws.com/latest/install
```

```
chmod +x ./install
```

```
sudo ./install auto
```

```
sudo service codedeploy-agent status
```

- Create appspec.yml and scripts/restart_tomcat.sh
- Add, commit and push.
- CodeDeploy
 - Go to AWS CodeDeploy
 - Left menu Applications » Create application

- Fill in:
 - Application name: devops-webapp
 - Compute platform: EC2/On-premises
- Click Create.
- After creation create deployment
- Application: devops-webapp
- Deployment group: devops-webapp-group
- Revision type: My application is stored in S3
- AWS » CodeBuild » Build history » click last successful build
- Go to the artifacts section
- Copy the source
- Deploy Code deploy
- We have our web-app deployed and automated